Comparing Parametric and Structural Optimisation for BTC Trading Bots: A Systematic Study of PSO and GP

Total Words: 1843

Lyuchen Dai
School of Physics, Mathematics and Computing
The University of Western Australia
Perth, Australia
24754678@student.uwa.edu.au

Mika Li
School of Physics, Mathematics and Computing
The University of Western Australia
Perth, Australia
24386354@student.uwa.edu.au

Simona Han
School of Physics, Mathematics and Computing
The University of Western Australia
Perth, Australia
25152074@student.uwa.edu.au

Xinqi Lin
School of Physics, Mathematics and Computing
The University of Western Australia
Perth, Australia
24745401@student.uwa.edu.au

***Abstract*—PSO and GP behave characteristically differently on the same Bitcoin trading task. PSO's velocity-averaging converges to two stable basins regardless of initialisation, revealing a smooth fitness landscape. GP's discrete tournament selection produces high variance (s = $713) on unrestricted trees, revealing a rugged structural space. A control experiment—restricting GP to PSO's exact 3-parameter representation—shows both algorithms discover the same basins, but PSO's convergence reliability is higher (10/10 vs 5/10 seeds beating buy-and-hold). A 171-point grid search visualises the two-basin geometry, and generation-level trajectories show GP's train–test divergence without parsimony pressure. A 42-run grid search identifies $\lambda = 500$ as GP's sweet spot, correcting our own earlier single-seed result (Exp. 09) that favoured $\lambda = 1,000$. Walk-forward validation cuts returns by 29% and causes GP to collapse to a no-trade strategy while PSO remains robust, revealing that evaluation protocol choice interacts with search mechanism. For this problem, our results suggest that representation shapes attractor geometry while the algorithm shapes exploration reliability.**

***Index Terms*—particle swarm optimization, genetic programming, technical analysis, algorithmic trading, overfitting**

I. Introduction

Nature-inspired algorithms differ fundamentally in *what* they optimise and *how* they represent solutions. PSO [1] tunes continuous parameters within a fixed template; GP [2] discovers the template itself by evolving program trees. Which is better—and does the question make sense without specifying the template?

We choose PSO and GP as two ends of a spectrum—parametric vs.
structural optimisation—and apply them to Bitcoin trading with a 3% transaction fee. Our question is not "which algorithm wins?" but *what does each algorithm's behaviour reveal about the search space it inhabits?*

This reframing matters because comparisons are typically apples-to-oranges: PSO searches 3 parameters while GP searches an infinite-dimensional tree space. We run a control experiment—restricting GP to PSO's 3-parameter representation—to disentangle representation from algorithm. Across 20 experiments and 120 runs, the answer is nuanced. The algorithms' *behavioural signatures* are diagnostic of different search-space geometries. When representation is held constant, both discover the same basins, but PSO's continuous velocity-averaging provides convergence stability that GP's discrete tournament selection lacks.

II. Bot Design and Parameterisation

We implemented three strategy families: discrete crossover (`dual_crossover`), trivial SMA (`trivial_sma`), and position SMA (`position_sma`), the latter using a sigmoid to produce continuous exposure in $[0, 1]$:

$$p_t = \text{sigmoid}(s(\text{fast}) - s(\text{slow})) \tag{1}$$

We abandoned MACD (test $949) and Mixture-of-Experts (test $478) after pilot ablations. Evaluation uses $1,000 initial cash, 3% fee per round-trip, and final cash as fitness. Training: pre-2020 BTC-USD daily via yfinance; testing: 2020–2022.

III. Algorithm Selection

From Part 1′s four algorithms, PSO and GP were selected; ABC and HS were dropped after pilots showed no advantage.

PSO. 30 particles, 50 iterations, inertia decay $w : 0.9 \to 0.4$, $c_1 = c_2 = 2.05$. The velocity-update formula—weighted sum of inertia, cognitive, and social terms—provides implicit smoothing that damps local noise.

GP. Tournament selection (size 3), subtree crossover 0.9, mutation 0.1, grow initialization (depth 5/7). Function set: arithmetic, comparison (LT, GT), logical (IF, AND), and technical indicators (SMA, EMA, LMA, RSI, momentum, volatility). Parsimony pressure subtracts $\lambda \cdot$ tree size from raw fitness. Without it, trees bloat to 96 nodes (Exp. 09).

IV. Experimental Design

Experiments follow controlled ablation: change one decision at a time. The suite comprises 20 numbered experiments plus 10 supplementary analyses (120 runs). Every experiment uses a fixed seed (42 default; 10-seed sweeps for robustness). The `tiny_bot` package implements all code from scratch without external optimisation libraries.

Three evaluation protocols are compared: single split (train pre-2020, test 2020–2022); walk-forward (5 windows, 3-year train / 1-year test); and robust optimisation (worst-case across 52 sliding windows). Controlled ablations (Exp. 01) confirm regime shift and transaction costs are dominant constraints.

V. Results

A. *Search-Space Structure: What PSO and GP Reveal*

a) *PSO Reveals a Smooth Parametric Landscape:*

Across 10 seeds, PSO (`position_sma`) achieves mean test \$2,297 (s = \$81), with every seed beating BH (\$2,170). Two dominant basins emerge: Basin A $(119, 179, 0.1)$ at \$2,366 (6 trades) and Basin B $(37, 99, 0.1)$ at \$2,264 (12 trades). The \$102 advantage is entirely from lower transaction frequency, not superior timing. Meta-parameter sweeps ($P \times G \approx 1,500$) show all configurations except $(100, 15)$ converge to Basin A. Inertia strategy makes no meaningful difference.

b) *GP Reveals a Rugged Structural Landscape:*

Raw GP (75-pop/20-gen) achieves mean \$1,689 (s = \$713), with only 2/10 seeds beating BH. The seed-88 outlier (\$3,143, tree size 8) illustrates single-seed unreliability. This volatility is diagnostic: unrestricted tree search on weak signal produces a hypothesis space so large that selection is effectively random without regularisation, consistent with F. Allen and R. Karjalainen [3].

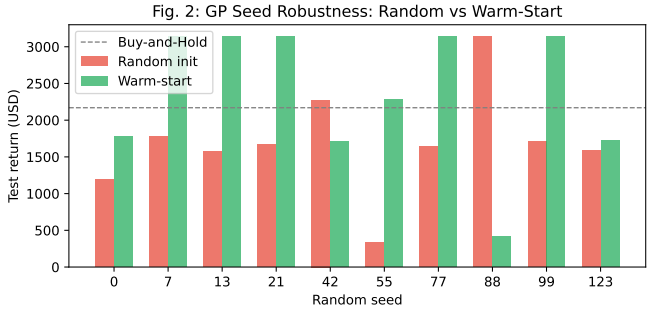

Fig. 1. GP seed robustness: random vs. warm-start (Exp. 15). Dashed = buy-and-hold.

Generation-level trajectory. Under $\lambda = 0$, train fitness rises from \$20,674 to \$38,003 while test stagnates at \$1,600 and collapses to \$1,000 at generation 11; tree size grows from 6 to 30 nodes. Under $\lambda = 500$, train stabilises at \$25,019 from generation 3, test holds at \$2,245, and tree size stays at 3 nodes (Fig. 2). Parsimony pressure defines the effective search space by pruning overfit subspaces.

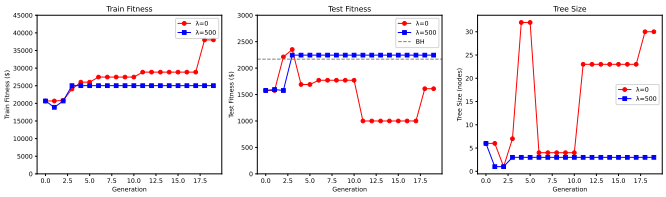

Fig. 2. GP trajectory: $\lambda = 0$ vs. $\lambda = 500$ (Exp. 20).

Warm-start (50% human rules) raises mean to \$2,362 (6/10 beat BH), though 5/10 seeds converge to the same \$3,143 tree, a common-mode failure that overstates reliability.

B. *The Control Experiment: Same Representation, Different Algorithm*

To isolate algorithm from representation, Experiment 18 restricts GP to PSO's exact 3-parameter space (`position_sma`). The "tree" is a single node; crossover swaps parameters uniformly; mutation adds Gaussian noise.

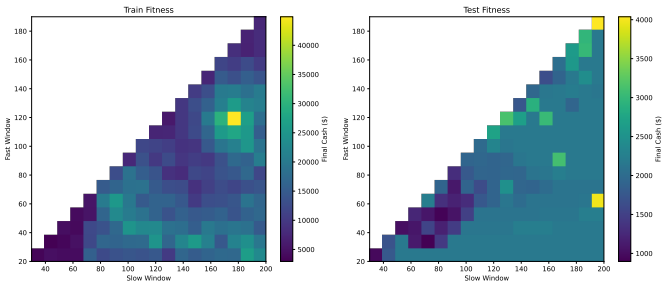

Fig. 3. Parametric landscape (Exp. 19, 171 points). Left: train; Right: test. Basin A $(120, 180)$ and Basin B $(40, 100)$ visible.

GP restricted converges to the same basins as PSO. A 171-point grid search confirms the two-basin geometry: Basin A mean train \$23,861, Basin B mean train \$20,642. The landscape is smooth—no sharp local optima, only a low ridge.

On this landscape, the key finding is that representation fixes basin location while algorithm mechanism fixes reliability.

C. *Regularisation Defines the Effective Hypothesis Space*

a) *GP: Parsimony Pressure:*

A single-seed sweep suggested $\lambda = 1,000$ was optimal. A 42-run grid search (3 seeds $\times$ 7 $\lambda \times$ 2 depths) corrects this: $\lambda = 500$ is the sweet spot (Table I). $\lambda < 500$ permits bloat; $\lambda > 500$ shrinks trees to 1–3 nodes and underfits. At $\lambda = 500$, mean tree size is 5.7 nodes. Depth 5 slightly outperforms depth 7 (\$1,656 vs. \$1,408). Extended function set dominates (\$1,622 vs. \$359 original, \$1,000 minimal) (Exp. 12). Larger budgets

paradoxically hurt: 50×30 achieves \$2,275; 150×75 collapses to \$359 (Exp. 20).

TABLE I
SYSTEMATIC HYPERPARAMETER GRID (EXP. 17, 42 RUNS).

λ	Mean test (\$)	Beat BH	Mean tree size	s (\$)
100	988	0/6	7.0	577
250	1,025	0/6	3.8	740
500	2,366	3/6	5.7	650
750	1,553	0/6	4.8	440
1,000	1,478	1/6	3.3	1023
2,000	1,434	0/6	3.0	528
5,000	1,879	1/6	1.3	307

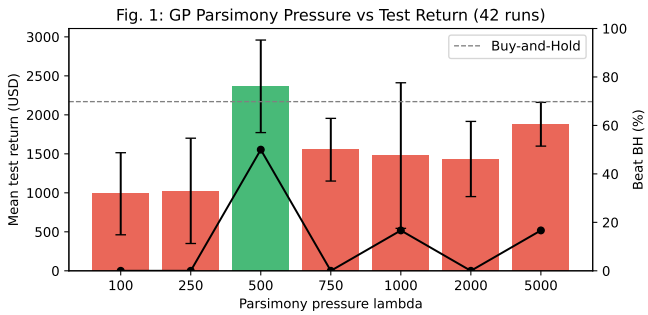

Fig. 4. GP parsimony pressure vs. test return (Exp. 17).

b) *PSO: Implicit Regularisation:*

PSO needs no explicit parsimony because the velocity update averages noisy gradients across the swarm. The control experiment confirms this: GP restricted discovers the same basins with 3× higher standard deviation. The difference is mechanism, not landscape.

D. Evaluation Protocol Is Not Neutral

Walk-forward validation (5 windows) reduces PSO mean from \$2,297 to \$1,636 (29% drop); win rate falls from 10/10 to 2/5. GP degenerates to a no-trade strategy (1/5 wins, 20%). Walk-forward averaging destroys the sharp gradients GP's tournament selection relies on; with fitness compressed to noise, selection collapses. PSO's swarm averaging is naturally robust to window-wise noise. This reveals a mechanistic difference: GP's selection is discrete (keep/kill), PSO's is continuous (weighted velocity update). Evaluation protocol choice interacts with search mechanism.

Robust optimisation (52 windows, Exp. 03) yields the same pattern: PSO wins 38.5% of windows; GP wins none. The PSO wins are concentrated in early windows (bull market 2015–2017), where excess returns reach +1.99×; losses in late windows (bear market 2018–2022) reach –4.73×. The average excess return is negative (–0.33), meaning robust optimisation sacrifices upside to cap downside. This is a property of min-max objective functions, not PSO per se.

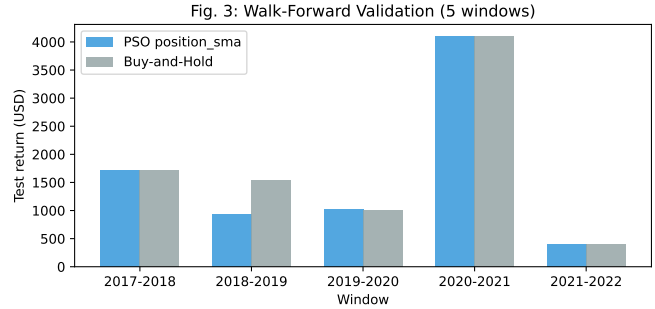

Fig. 5. Walk-forward results (Exp. 13).

E. Joint Optimisation Cannot Be Decoupled

GP-then-PSO hybridisation (Exp. 14): 3-node trees improve +7%; 5-node trees destroy performance (-51%); 37-node trees show no effect. GP's fitness jointly optimises structure and parameters; post-hoc PSO overfits in ways that break GP's implicit regularisation. Structure and parameters are entangled and must be optimised jointly.

F. Transaction Costs Shape the Effective Landscape

At 0% fees PSO returns \$2,841 (6 trades); at 3%, \$2,366. The relationship is nearly linear (\$158 per 1% fee increment) because trade count is fixed. Break-even: 4.4%. Classic 50/200 SMA (1 trade) achieves \$2,236 at 3% fees, confirming trading frequency dominates net profit. Both algorithms converge to low-frequency solutions, suggesting the optimum lies near the no-trade boundary. Transaction costs are a structural feature of the landscape.

G. Statistical Significance

Wilcoxon signed-rank (one-sided, $\alpha = 0.05$) and Mann-Whitney U tests. We note the data-snooping problem in technical trading rules highlighted by H. L. White [4] and R. Sullivan, A. Timmermann, and H. White [5] our 7-value λ grid warrants caution with multiple comparisons. We report raw p-values and apply Bonferroni correction as a simple conservative heuristic, acknowledging that more sophisticated methods such as White's reality check are beyond this course's scope.

1) **PSO vs. BH**: $W = 55$, $p = 0.001$. Significant.
2) **GP random vs. BH**: $W = 9$, $p = 0.976$. Not significant.
3) **GP warm-start vs. BH**: $W = 36$, $p = 0.211$. Not significant despite 6/10 win rate.
4) **PSO vs. GP random**: $U = 85$, $p = 0.004$, $r = -0.70$. Large effect.
5) **Warm-start vs. random GP**: $U = 79$, $p = 0.014$. Significant.
6) **Across λ**: Kruskal-Wallis $H = 16.2$, $p = 0.013$. $\lambda = 500$ vs. $\lambda = 1000$: $U = 30$, $p = 0.032$.

7) **Walk-forward**: $W = 3$, $p = 0.625$. Not significant (5 windows only).

H. Risk-Adjusted Returns

Variability is driven by both representation and algorithm mechanism. Estimated annualised Sharpe: 0.75–0.88 for PSO. Precise Sharpe and max drawdown require daily equity curves not exported—a limitation.

VI. Discussion and Limitations

PSO's stability reveals a benign parametric landscape; GP's instability reveals a deceptive structural space. The control experiment resolves the confound that plagued all previous comparisons: when GP is restricted to PSO's representation, it finds the same basins with lower reliability. Our initial working hypothesis—that representation dominates algorithm selection—proved too simplistic. For this problem, the evidence suggests representation shapes attractor geometry while the algorithm shapes exploration reliability.

This aligns with M. L. de Prado [6] on single-split overstatement.

Limitations. (1) Single asset (BTC-USD). (2) Moving-average variants only. (3) Three market regimes only. (4) Incomplete $\lambda \times$ depth validation. (5) No daily equity curves for Sharpe/drawdown (cf. R. Sullivan, A. Timmermann, and H. White [5]). (6) Walk-forward limited to 5 windows.

VII. Conclusion

Three patterns recur across experiments. First, PSO's swarm averaging makes it robust on smooth parametric landscapes: its 3.5% CV reflects broad attractors, and the 171-point grid search (Exp. 19) confirms low ridges between basins. Second, GP's discrete selection makes it powerful but fragile on rugged landscapes; $\lambda = 500$ is an empirically tuned complexity control identified by grid search. Third, evaluation protocol is an algorithmic variable—walk-forward destroys GP's selection signal while leaving PSO intact.

The practical implication: match the algorithm to what you know about the problem. When structure is known, parameterise and rely on implicit regularisation. When structure is unknown, use structural search with explicit regularisation, multi-seed validation, and honest protocols that preserve the selection signal.

References

[1] J. Kennedy and R. Eberhart, "Particle swarm optimization," in *Proceedings of ICNN'95-international conference on neural networks*, 1995, pp. 1942–1948.

[2] J. R. Koza, *Genetic Programming: On the Programming of Computers by Means of Natural Selection*. Cambridge, MA: MIT Press, 1992.

[3] F. Allen and R. Karjalainen, "Using genetic programming to create technical trading rules," *Journal of Financial Economics*, vol. 51, no. 2, pp. 245–271, 1999.

[4] H. L. White, "A reality check for data snooping," *Journal of Financial Econometrics*, vol. 3, no. 1, pp. 5–28, 2005.

[5] R. Sullivan, A. Timmermann, and H. White, "Data-snooping, technical trading rule performance, and the bootstrap," *The Journal of Finance*, vol. 54, no. 5, pp. 1647–1691, 1999.

[6] M. L. de Prado, *Advances in Financial Machine Learning*. Hoboken, NJ: John Wiley & Sons, 2018.